

High Performance Computing (HPC)

Table of Contents

Sr. No.	Title	Page
1	Introduction	2
2	HPC Hardware Components	5
3	Networking in HPC	15
4	HPC Storage Architecture	22
5	RAID Technology	23
6	JBOD	28
7	File Systems	32
8	HPC Software Stack	36
9	Job Schedulers	40
10	MPI	45
11	Benchmarking HPC Cluster	50
12	Physical HPC Cluster Setup	57
13	GPU Computing in HPC	70
14	High Availability (HA) Clustering	76
15	Summary	79

Chapter 1: Introduction to HPC

1.1 What is High Performance Computing?

High Performance Computing (HPC) refers to the practice of aggregating computing power in a way that delivers much higher performance than a typical desktop computer or workstation, in order to solve large, complex problems in science, engineering, research, and industry.

HPC systems — commonly called clusters or supercomputers — combine hundreds or thousands of processors working in parallel to process data and perform calculations at extremely high speeds. Tasks that would take years on a standard computer can be completed in hours or even minutes on an HPC system.

At its core, HPC is about three things:

Pillar	Description
Compute	Many processors working simultaneously in parallel
Memory	Large amounts of RAM to hold massive datasets
Storage	High-speed, high-capacity shared storage accessible by all nodes

1.2 Why HPC?

Modern scientific and engineering problems generate enormous amounts of data and require massive computational power that a single computer simply cannot provide. Some real-world examples include:

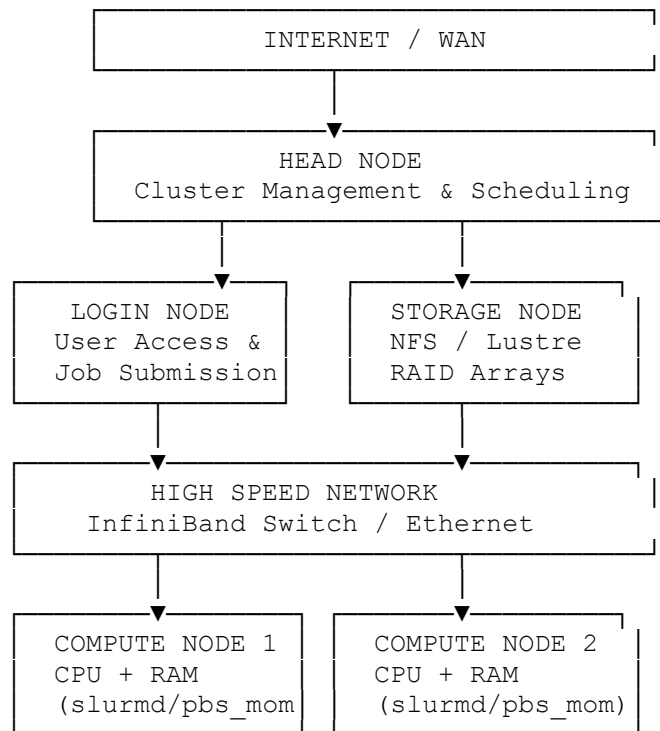
Domain	HPC Use Case	Why HPC is Needed
Computational Fluid Dynamics	Airflow simulation over aircraft	Millions of mesh points calculated simultaneously
Finite Element Analysis	Crash simulation, stress testing	Complex matrix equations across thousands of elements
Weather Forecasting	Climate and atmospheric modeling	Petabytes of sensor data processed in real time
Drug Discovery	Molecular dynamics simulation	Billions of atomic interactions per second
Artificial Intelligence	Training large neural networks	Massive parallel matrix operations on GPUs
Oil & Gas	Seismic data processing	3D subsurface imaging from sensor arrays
Genomics	DNA sequencing and analysis	Comparing billions of base pairs
Financial Modeling	Risk analysis, option pricing	Monte Carlo simulations with millions of scenarios

1.3 HPC vs Traditional Computing

Feature	Laptop / Workstation	HPC Cluster
Processors	1 CPU, 4-16 cores	Hundreds to thousands of CPUs
Memory	8-64 GB	Terabytes
Storage	1-4 TB local	Petabytes shared
Network	Gigabit Ethernet	InfiniBand (200 Gbps)
Operating System	Windows / Linux	Linux (RHEL, Rocky, CentOS)
Job Management	Manual	Automated scheduler (Slurm, PBS)
Use Case	General purpose	Specialized scientific computing
Cost	\$1,000 - \$5,000	\$100,000 - \$100,000,000+

1.4 HPC Cluster Overview

An HPC cluster is a collection of interconnected computers (nodes) that work together as a unified system. Each node type has a specific role:



Chapter 2: HPC Hardware Components

2.1 Head Node (Master Node)

The head node is the central management server and brain of the HPC cluster. All cluster management, job scheduling, and administrative operations are performed from the head node. It is the only node that typically has direct access to the internet or corporate network.

Responsibilities:

- Runs cluster management software (xCAT, HPCM)
- Runs the job scheduler (Slurm, OpenPBS)
- Hosts DNS, DHCP, NTP, and NFS services
- Manages node provisioning and OS deployment via PXE
- Monitors cluster health and resource utilization
- Manages user accounts and authentication services

Typical Hardware Specification:

Component	Specification
CPU	2x Intel Xeon Gold / AMD EPYC (16-32 cores each)
RAM	256-512 GB DDR4/DDR5
Storage	2x 1.92TB SSD in RAID 1 (OS)
Network	2x 25GbE + 1x 1GbE (IPMI)
Power	Redundant PSUs

In production clusters, two head nodes are deployed in active-passive HA (High Availability) configuration to prevent a single point of failure. If the primary head node fails, the secondary takes over automatically.

2.2 Login Node (Access Node / Front-End Node)

The login node is the gateway through which users access the cluster. Users SSH into the login node to submit jobs, compile code, manage their data, and interact with the cluster. Users never SSH directly into compute nodes — all job execution goes through the scheduler.

Responsibilities:

- Accepts user SSH connections from outside the cluster
- Provides interactive shell environment for users
- Job submission to the scheduler (sbatch, qsub)
- Code compilation, debugging, and testing
- File management and data staging

Typical Hardware Specification:

Component	Specification
CPU	2x Intel Xeon (16-32 cores each)
RAM	128-256 GB
Storage	Local OS disk (boots from local)
Network	25 GbE cluster + 1 GbE management
NFS Mounts	/home, /soft, /opt from storage node

The login node should never be used to run compute jobs directly. Heavy computation on the login node degrades the experience for all users and may destabilize the node.

2.3 Compute Nodes

Compute nodes are the workhorses of the HPC cluster. They execute the actual computational jobs assigned by the scheduler. A cluster can have anywhere from a few to thousands of compute nodes, all working in parallel.

Types of Compute Nodes:

Type	Description	Typical Use Case
Standard CPU Node	Dual socket server, 32-128 cores	General simulation, FEA, CFD
GPU Node	CPUs + NVIDIA GPUs	AI/ML, deep learning, CUDA
High Memory Node	Large RAM (1-6 TB)	In-memory databases, genomics
Fat Node	Many cores + large RAM	Memory-intensive simulations
Many-Core Node	Intel Xeon Phi	Vectorized HPC workloads

Typical CPU Compute Node Hardware:

Component	Specification
CPU	2x Intel Xeon / AMD EPYC (64-128 cores total)
RAM	256-512 GB DDR4/DDR5
Local Storage	1x 480GB SSD (OS + scratch)
Cluster Network	1x InfiniBand HDR (200 Gbps)
Management Network	1x 1 GbE
Power	Redundant PSUs

2.4 Storage Node

The storage node provides centralized shared storage to all other nodes in the cluster. All nodes mount the storage node's filesystems, giving users consistent access to their data regardless of which compute node their job runs on.

Responsibilities:

- Hosts and exports shared filesystems (/home, /soft, /opt, /scratch)
- Manages RAID arrays for data protection and performance
- Provides NFS or Lustre services to all cluster nodes
- Handles all cluster I/O traffic

Typical Hardware Specification:

Component	Specification
CPU	2x Intel Xeon (for NFS/Lustre processing)
RAM	128-256 GB (large RAM improves I/O cache)
OS Disk	2x SSD RAID 1
Data Disks	12-24x SAS/NL-SAS HDDs or SSDs
Network	2x 25 GbE (bonded for high throughput)
HBA	SAS HBA for external disk shelves

2.5 MDT Server (Metadata Target — Lustre)

In a Lustre parallel filesystem, the MDT server stores all filesystem metadata. Metadata includes file names, directory structure, permissions, ownership, timestamps, and the location mapping of where file data is stored across the OSS servers. The MDT does not store actual file data.

Responsibilities:

- Handles all namespace operations (ls, mkdir, stat, chmod, chown)
- Tracks which OSS/OST stores each file's data blocks
- Manages distributed file locking
- Handles file creation and deletion operations

Critical Performance Note: Every single file access in Lustre begins with a metadata lookup on the MDT. A slow or overloaded MDT degrades the performance of the entire filesystem for all users, even if the OSSes have plenty of capacity. NVMe SSDs are strongly recommended for MDT storage.

2.6 OSS Server (Object Storage Server — Lustre)

The OSS (Object Storage Server) stores the actual file data in Lustre. Multiple OSS servers work in parallel, each managing one or more OSTs (Object Storage Targets — the physical disk groups). Files are striped across multiple OSTs to enable parallel I/O from all compute nodes simultaneously.

Responsibilities:

- Stores actual file data on OSTs (disk pools)
- Handles read and write requests from compute node clients
- Multiple OSSes work simultaneously for high aggregate bandwidth
- Each OSS manages one or more OSTs (RAID groups of disks)

Performance Scaling Example:

```
1 OSS  × 10 GB/s per OSS  = 10 GB/s total bandwidth
10 OSS × 10 GB/s per OSS  = 100 GB/s total bandwidth
50 OSS × 10 GB/s per OSS  = 500 GB/s total bandwidth
```

This linear scalability is impossible with NFS which is limited to a single server's bandwidth.

2.7 PDU (Power Distribution Unit)

A PDU distributes electrical power from the facility power supply to all rack-mounted equipment. In HPC clusters, intelligent managed PDUs provide remote power monitoring and control capabilities, which are essential for managing large numbers of servers.

Types of PDUs:

Type	Features	Use Case
Basic PDU	Simple power strip, no monitoring	Small test clusters
Metered PDU	Per-outlet current and power monitoring	Power usage tracking
Switched PDU	Remote on/off control per outlet	Remote power cycling
Intelligent PDU	Monitoring + switching + alerting + energy reporting	Production HPC

Key metrics monitored by intelligent PDUs:

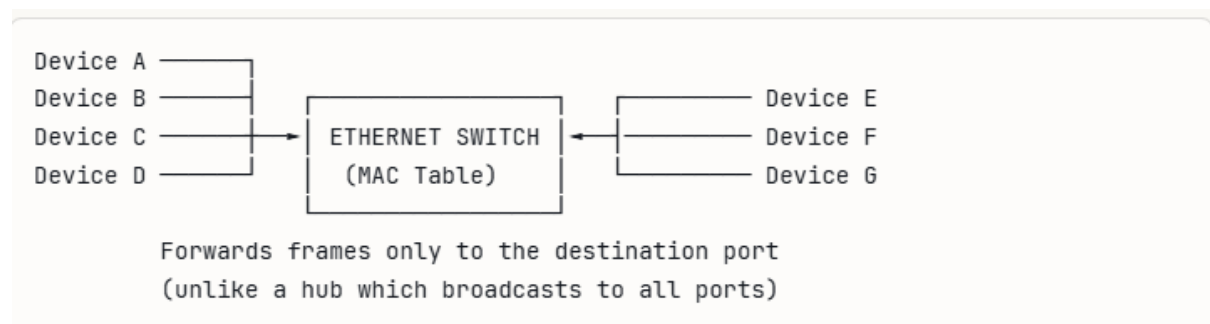
- Per-outlet current (amps) and power (watts)
- Total rack power consumption and trending
- Phase-level current for 3-phase power
- Inlet temperature and humidity
- Remote power cycling for hung/unresponsive nodes

Best Practice: Always deploy two PDUs per rack on separate facility circuits (Circuit A and Circuit B). Connect the primary PSU of each server to PDU-A and the redundant PSU to PDU-B. This ensures no single circuit failure can take down an entire rack.

2.7 Ethernet Switch

An Ethernet switch is a networking device that connects multiple devices within a network and forwards data packets between them. It operates at Layer 2 (Data Link) of the OSI model using MAC addresses to direct traffic, and optionally at Layer 3 (Network) for routing between subnets.

In HPC clusters, Ethernet switches are used for management networks, storage (NFS) traffic, and user-facing connections. While they are slower than InfiniBand, they are less expensive, simpler to manage, and sufficient for all non-MPI traffic.



Switch Types Used in HPC

Switch Type	Layer	Typical Speed	Role in HPC
Access Switch	L2	1 GbE	Connects IPMI/BMC ports of all servers
Distribution Switch	L2/L3	10/25 GbE	Aggregates access switches, NFS traffic
Core Switch	L3	100 GbE	Cluster backbone, routing between networks
Top-of-Rack (ToR)	L2/L3	25/100 GbE	One switch per rack, connects all servers in that rack

3.2.3 Key Switch Specifications

Specification	Description	Why It Matters
Port Count	Number of access ports (24, 48, etc.)	Determines how many servers can connect
Port Speed	1G / 10G / 25G / 100G per port	Higher = more bandwidth per server
Uplink Speed	40G / 100G uplink ports	Carries aggregated traffic to core
Switching Capacity	Total bandwidth (Tbps)	Must exceed sum of all port speeds
Forwarding Rate	Packets per second (Mpps)	Determines latency under load
Buffer Size	MB of packet buffer	Prevents packet drops during bursts
Oversubscription	Access bandwidth ÷ uplink bandwidth	Lower is better for HPC
Management	CLI, SNMP, REST API	Remote administration
Redundancy	Redundant PSU, fans	High availability

2.8 InfiniBand (IB) Switch

InfiniBand (IB) is a high-speed, low-latency serial networking technology designed specifically for high-performance computing. It was developed in the late 1990s as a replacement for PCI buses for server interconnects, and has since become the dominant technology for MPI communication in HPC clusters.

Unlike Ethernet which uses TCP/IP — a protocol with significant software overhead — InfiniBand uses RDMA (Remote Direct Memory Access), which allows data to be transferred directly between the memory of two remote nodes without involving the CPU or operating system of either node. This dramatically reduces both latency and CPU overhead.

Standard Ethernet (TCP/IP):

App → OS → TCP Stack → NIC → Network → NIC → TCP Stack → OS → App
(many CPU interrupts, copies, and context switches)

InfiniBand (RDMA):

App ————— App
(data moves directly between application memory)
(CPU and OS are NOT involved in the transfer)

InfiniBand Generations

InfiniBand has evolved through multiple generations, each doubling the bandwidth of the previous:

Generation	Full Name	Per-Port Bandwidth	Typical Connector	Status
SDR	Single Data Rate	8 Gbps (1x)	QSFP	Legacy
DDR	Double Data Rate	16 Gbps (2x)	QSFP	Legacy
QDR	Quad Data Rate	32 Gbps (4x)	QSFP	Older systems
FDR	Fourteen Data Rate	56 Gbps (14x)	QSFP+	Still found
EDR	Enhanced Data Rate	100 Gbps	QSFP28	Common
HDR	High Data Rate	200 Gbps	QSFP56	Modern HPC
NDR	Next Data Rate	400 Gbps	OSFP	Latest generation
XDR	Extended Data Rate	800 Gbps	OSFP	Emerging

InfiniBand vs Ethernet Comparison

Specification	InfiniBand HDR	100 GbE Ethernet	25 GbE Ethernet
Bandwidth per port	200 Gbps	100 Gbps	25 Gbps
Latency	~600 nanoseconds	~5 microseconds	~5 microseconds
Protocol	RDMA native	TCP/IP (RDMA via RoCE)	TCP/IP
CPU overhead	Very low	Moderate to high	Moderate
MPI performance	Excellent	Good (with RoCE)	Acceptable
Lustre support	Excellent	Good	Acceptable
Cost	Higher	Moderate	Lower
Complexity	Higher	Lower	Lower
Primary HPC use	MPI + Lustre	MPI (medium clusters)	NFS storage

InfiniBand Components

Component	Description
HCA (Host Channel Adapter)	InfiniBand NIC installed in compute/storage nodes
Switch	IB switch that connects multiple HCAs
Subnet Manager (SM)	Software that manages IB network topology (runs on head node or switch)
RDMA	Protocol enabling direct memory access between nodes
OFED	OpenFabrics Enterprise Distribution — IB driver stack for Linux

2.8 Cables Used in HPC

Ethernet Cables

Category	Maximum Speed	Maximum Frequency	Maximum Distance	Typical Use in HPC Environments
CAT5	100 Mbps	100 MHz	100 meters	Obsolete; rarely used in modern systems
CAT5e	1 Gbps	100 MHz	100 meters	Legacy management networks
CAT6	1 Gbps (100m) / 10 Gbps (up to 55m)	250 MHz	100 meters (1G), 55 meters (10G)	IPMI / BMC management networks
CAT6A	10 Gbps	500 MHz	100 meters	10 GbE management and service networks
CAT7	10 Gbps	600 MHz	100 meters	Shielded cabling used in data centers
CAT8	25 / 40 Gbps	2000 MHz	30 meters	High-speed short-distance data center connections

InfiniBand (IB) Cables

InfiniBand cables carry the high-speed RDMA traffic between compute nodes, storage nodes, and InfiniBand switches. They are fundamentally different from Ethernet cables in both physical construction and electrical signaling.

Active Optical Cables (AOC)

AOC cables use fiber optic technology inside a cable assembly with QSFP connectors at both ends. The optical-to-electrical conversion happens inside the connector housing.

Passive Copper Cables (DAC — Direct Attach Copper)

DAC cables use copper wires with fixed QSFP connectors. No optical conversion — purely electrical. Shorter range but lower cost.

Cable Summary

Cable Type	Connector	Typical Speed	Maximum Length	Typical Use in HPC Environments
CAT6 Copper	RJ45	1 GbE	100 m	IPMI / BMC and management network
CAT6A Copper	RJ45	10 GbE	100 m	10 GbE service or management network
SFP28 DAC (Direct Attach Copper)	SFP28	25 GbE	5 m	Server to Top-of-Rack (ToR) switch
SFP28 AOC (Active Optical Cable)	SFP28	25 GbE	100 m	Server to remote switch
QSFP28 DAC	QSFP28	100 GbE	5 m	Short switch uplinks within rack
QSFP28 AOC	QSFP28	100 GbE	100 m	Switch uplinks between racks
QSFP28 SR4 + MPO Fiber	QSFP28 + MPO	100 GbE	100 m (OM4)	Fiber switch uplinks
QSFP28 to 4×SFP28 Breakout Cable	QSFP28 + 4×SFP28	4 × 25 GbE	5 m	Fan-out from switch to four servers
InfiniBand HDR DAC	QSFP56	200 Gbps	3 m	In-rack InfiniBand node connections
InfiniBand HDR AOC	QSFP56	200 Gbps	100 m	Inter-rack InfiniBand connections
InfiniBand HDR to 2×EDR Splitter	QSFP56 + 2×QSFP28	2 × 100 Gbps	2 m	HDR switch port split to two EDR servers
InfiniBand NDR AOC	OSFP	400 Gbps	100 m	Next-generation InfiniBand uplinks
Multi-mode Fiber (OM4)	LC / MPO	10–100 GbE	Up to 400 m	Long intra-data-center runs
Single-mode Fiber (OS2)	LC	10–100 GbE	Up to 40 km	Campus or inter-building connections

Chapter 3: Networking in HPC

3.1 Overview of HPC Networking

HPC clusters typically deploy multiple separate networks for different purposes. Separating network traffic ensures that management operations do not interfere with job traffic, and that high-speed job communication gets dedicated bandwidth.

Network Type	Speed	Purpose
IPMI / BMC Management	1 GbE	Out-of-band hardware management
Administration Network	1 GbE	OS provisioning, xCAT, DNS, NTP
Service / Data Network	10/25 GbE	NFS storage traffic, user SSH
Application Network	InfiniBand 100-200 Gbps	MPI job communication, Lustre

3.2 Ethernet Switches

Ethernet switches are used for management, administration, and data networks in the cluster. They operate at Layer 2 (data link) and Layer 3 (network/routing) of the OSI model.

Role in HPC clusters:

- Management switch connects all IPMI/BMC ports for out-of-band hardware control
 - Service switch carries NFS storage traffic and user SSH connections
 - Some clusters use 25/100 GbE Ethernet instead of InfiniBand for the application network
-

3.3 InfiniBand Switch

InfiniBand (IB) is a high-speed, low-latency networking technology used in HPC clusters primarily for MPI communication between compute nodes and for Lustre filesystem traffic. It provides significantly higher bandwidth and dramatically lower latency than standard Ethernet.

InfiniBand Generations:

Generation	Abbreviation	Bandwidth per Port	Typical Use
Single Data Rate	SDR	8 Gbps	Legacy
Double Data Rate	DDR	16 Gbps	Legacy
Quad Data Rate	QDR	32 Gbps	Older systems
Fourteen Data Rate	FDR	56 Gbps	Common
Enhanced Data Rate	EDR	100 Gbps	Current
High Data Rate	HDR	200 Gbps	Modern HPC
Next Data Rate	NDR	400 Gbps	Latest

InfiniBand vs Ethernet comparison:

Specification	InfiniBand HDR	100 GbE Ethernet
Bandwidth	200 Gbps	100 Gbps
Latency	~600 nanoseconds	~5 microseconds
Protocol	RDMA native	TCP/IP (RDMA via RoCE)
CPU Overhead	Very low	Higher
Cost	Higher	Lower
Primary Use	MPI, Lustre	General networking

RDMA (Remote Direct Memory Access): InfiniBand supports native RDMA, which allows data to be transferred directly between the memory of two nodes without involving the CPU of either node. This dramatically reduces latency and CPU overhead for MPI applications.

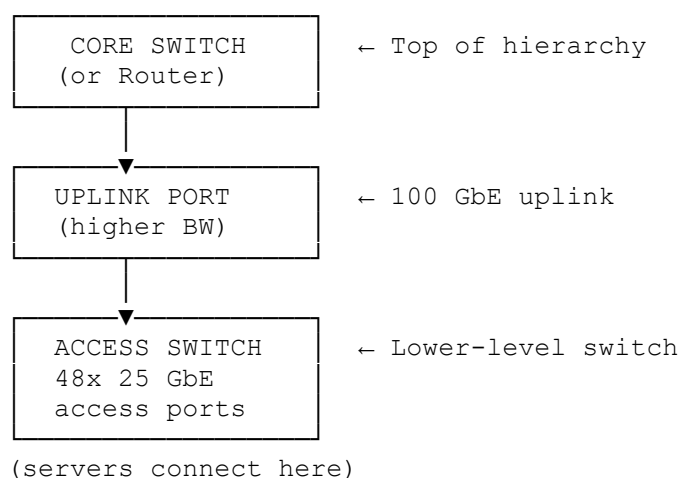
Common vendors: NVIDIA (formerly Mellanox)

3.4 Uplinks

3.4.1 What is an Uplink?

An uplink is a network connection that links a lower-level switch to a higher-level switch, router, or core network device. The term comes from hierarchical network design — data flows "up" from edge devices toward the core of the network.

In the context of networking, an uplink carries aggregated traffic from multiple downstream ports to the next level of the network hierarchy. Because an uplink must carry the combined traffic of all devices connected to the lower switch, uplinks are always configured with higher bandwidth than individual access ports.



Key principle — Oversubscription: If 48 servers each have a 25 GbE connection to an access switch, the total possible traffic is $48 \times 25 = 1,200$ Gbps. If the uplink is 100 GbE, the oversubscription ratio is 12:1. In HPC clusters, low oversubscription ratios (ideally 1:1 for InfiniBand) are critical for MPI performance.

3.4.2 Uplink Types

Uplink Type	Speed	Use Case
Single 10 GbE	10 Gbps	Small management networks
Single 25 GbE	25 Gbps	Service network uplinks
Single 100 GbE	100 Gbps	Core uplinks
LACP Bonded (2x25 GbE)	50 Gbps effective	Aggregated uplinks
LACP Bonded (4x25 GbE)	100 Gbps effective	High availability uplinks
InfiniBand HDR	200 Gbps	HPC application network uplinks

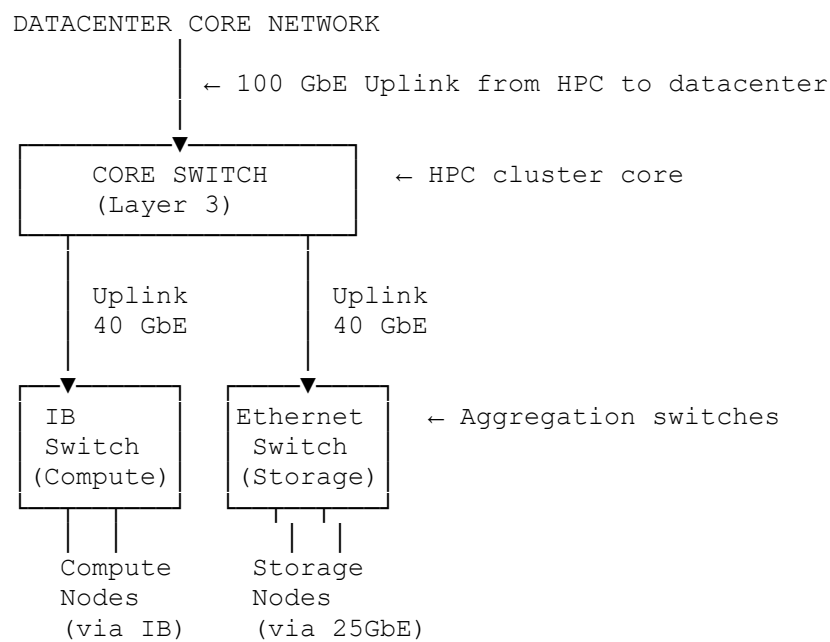
3.4.3 Uplinks in HPC Cluster Network Design

In an HPC cluster, uplinks appear at multiple levels of the network hierarchy:

Level 1 — Server to Access Switch: Each server connects its NIC to an access switch port. This is not technically an uplink but a standard access connection.

Level 2 — Access Switch to Aggregation/Core Switch (Uplink): Access switches connect upward to aggregation or core switches via uplink ports. This is where traffic from multiple servers is aggregated.

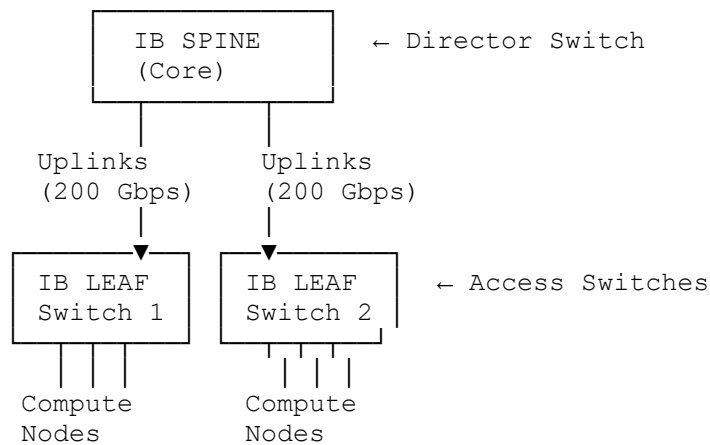
Level 3 — Aggregation Switch to Core Router (Uplink): Aggregation switches connect to the core router or the datacenter network fabric via high-speed uplinks.



3.4.4 Uplinks in InfiniBand Networks

In InfiniBand networks used for MPI, uplinks connect multiple IB leaf switches to an IB spine switch (director switch) in a fat-tree topology. The fat-tree topology is the dominant InfiniBand design in HPC because it provides non-blocking or low-blocking bandwidth between any two nodes.

Fat-Tree Topology:



In a non-blocking fat-tree, the uplink bandwidth equals the total downlink bandwidth — every node can communicate with every other node at full line rate simultaneously. This is critical for MPI collective operations (MPI_Allreduce, MPI_Bcast) that involve all nodes at once.

3.4.5 Link Aggregation (LACP Bonding) on Uplinks

To increase uplink bandwidth and provide redundancy, multiple physical uplink cables are bonded together using LACP (Link Aggregation Control Protocol), also called port bonding or NIC teaming.

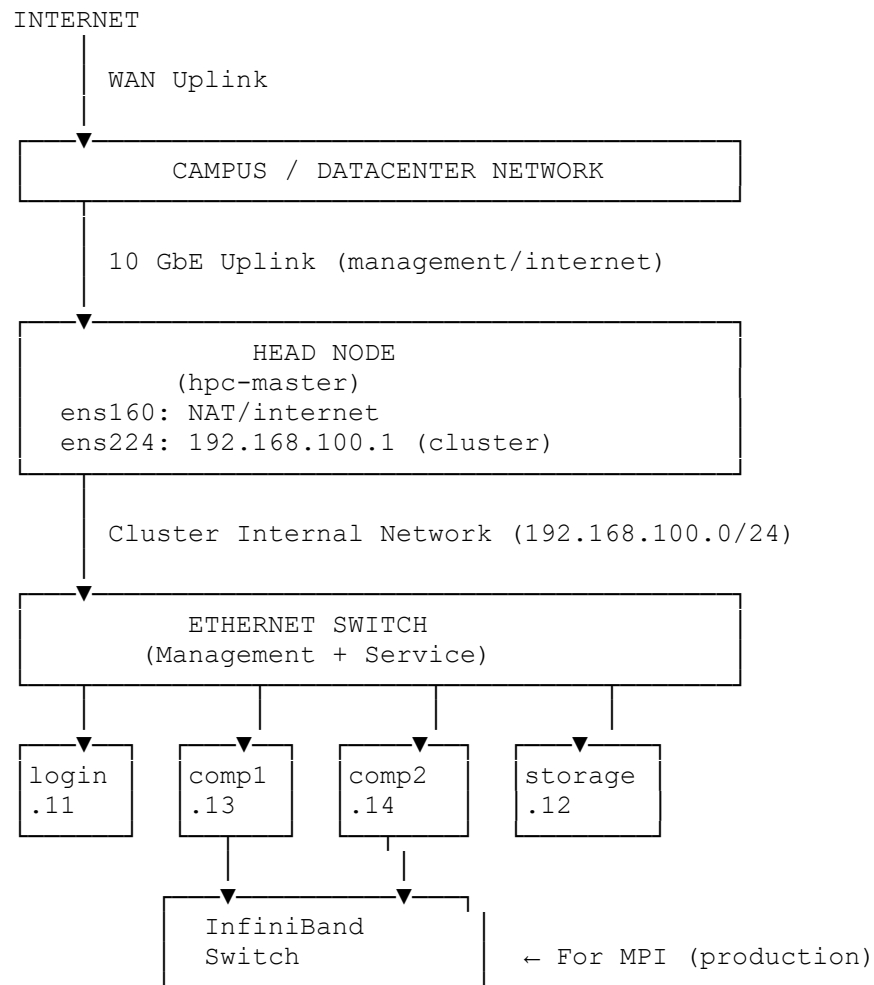
```
# Configure bonded uplink on Linux (for NFS storage traffic)
nmcli con add type bond ifname bond0 bond.options "mode=802.3ad"
nmcli con add type ethernet ifname ens224 master bond0
nmcli con add type ethernet ifname ens225 master bond0
nmcli con modify bond0 ipv4.addresses 192.168.100.12/24
nmcli con up bond0
```

Benefits of LACP bonding on uplinks:

- Doubles (or more) available bandwidth
 - Provides automatic failover if one cable fails
 - Load balances traffic across multiple physical links
 - Transparent to connected devices
-

3.4.6 Summary — HPC Network Design

A complete HPC cluster network design with uplinks:



Chapter 4: HPC Storage Architecture

4.1 Storage Hierarchy in HPC

HPC clusters use multiple tiers of storage, each optimized for different purposes:

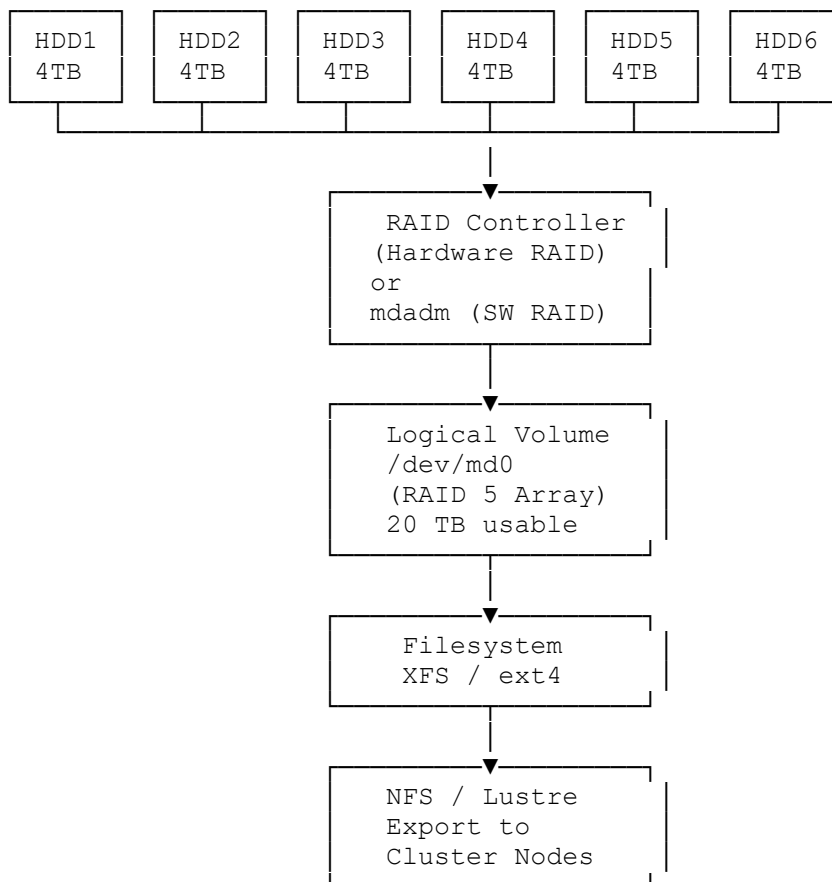
Tier	Type	Speed	Capacity	Purpose
Tier 0	NVMe SSD / Burst Buffer	Highest	Small	Checkpoint, temporary
Tier 1	Parallel FS (Lustre)	Very High	Large	Job scratch, simulation data
Tier 2	NFS (SSD-backed)	High	Medium	Home dirs, software
Tier 3	NFS (HDD-backed)	Moderate	Large	Archives, shared data
Tier 4	Tape / Object Store	Low	Massive	Long-term archive

4.2 Storage Disk Pool and Connection to RAID

A disk pool is a collection of physical drives grouped together to provide storage capacity. In HPC, disk pools are protected and organized using RAID technology.

How physical disks connect to RAID and filesystem:

PHYSICAL DISK POOL



Chapter 5: RAID Technology

5.1 What is RAID?

RAID (Redundant Array of Independent Disks) is a technology that combines multiple physical disks into a single logical unit to provide data redundancy, improved performance, or both. The goal is to protect data from disk failures and to improve I/O throughput.

RAID works on three fundamental principles: Stripping, Mirroring and Parity

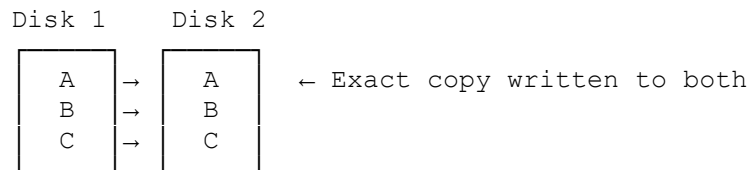
Principle	Description
Striping	Data is split and written across multiple disks simultaneously — improves performance
Mirroring	Data is duplicated on multiple disks — provides redundancy
Parity	Mathematical checksum stored across disks — allows reconstruction after failure

5.2 RAID 0 — Striping (Performance Only)

Disk 1	Disk 2	
A1	A2	← Block A split across both disks
B1	B2	← Block B split across both disks
C1	C2	

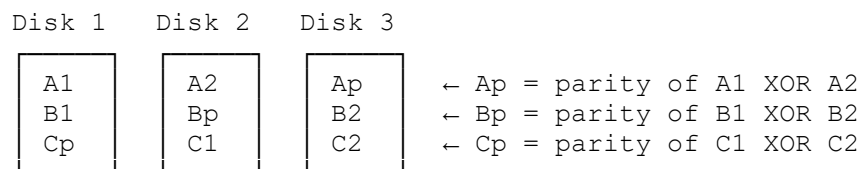
Property	Value
Minimum Disks	2
Usable Capacity	100% of total raw
Redundancy	None
Read Performance	Excellent (reads from all disks simultaneously)
Write Performance	Excellent (writes to all disks simultaneously)
Fault Tolerance	Zero — any single disk failure = complete data loss
Use Case	Temporary scratch space, non-critical high-speed storage

5.3 RAID 1 — Mirroring (Full Redundancy)



Property	Value
Minimum Disks	2
Usable Capacity	50% of total raw
Redundancy	Full mirror
Read Performance	Good (can read from either disk)
Write Performance	Moderate (must write to both disks)
Fault Tolerance	1 disk failure
Use Case	OS drives, critical small datasets, boot volumes

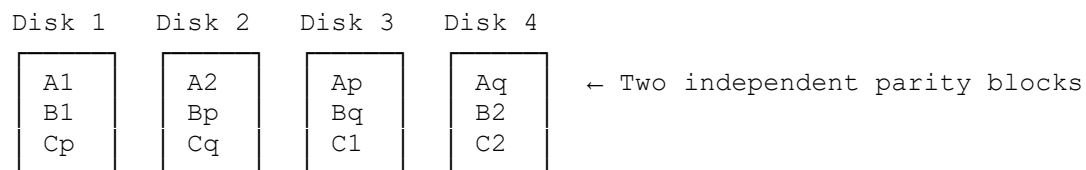
5.4 RAID 5 — Striping with Distributed Parity



Parity rotates across all disks

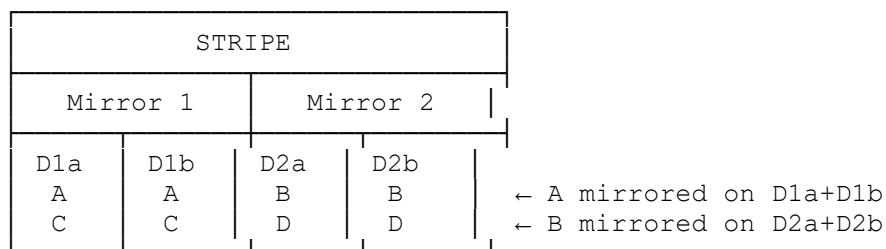
Property	Value
Minimum Disks	3
Usable Capacity	(N-1) × disk size
Redundancy	Single disk failure tolerance
Read Performance	Good
Write Performance	Moderate (parity calculation on every write)
Fault Tolerance	1 disk failure — array remains operational
Rebuild Risk	During rebuild after failure, a second failure = data loss
Use Case	General purpose storage, NAS, file servers

5.5 RAID 6 — Striping with Double Distributed Parity



Property	Value
Minimum Disks	4
Usable Capacity	$(N-2) \times \text{disk size}$
Redundancy	Double disk failure tolerance
Read Performance	Good
Write Performance	Slower than RAID 5 (two parity calculations)
Fault Tolerance	2 simultaneous disk failures
Use Case	Large disk arrays, archives, critical data where rebuild time is long

5.6 RAID 10 — Mirror + Stripe (RAID 1+0)



Property	Value
Minimum Disks	4
Usable Capacity	50% of total raw
Redundancy	1 disk per mirrored pair (can survive multiple failures)
Read Performance	Excellent
Write Performance	Excellent
Fault Tolerance	1 failure per mirror pair
Use Case	Databases, high transaction workloads, maximum performance + redundancy

5.7 RAID Comparison Summary

RAID	Min Disks	Usable Capacity	Fault Tolerance	Read	Write	Best For
RAID 0	2	100%	None	☆☆☆☆ ☆	☆☆☆☆☆	Speed only
RAID 1	2	50%	1 disk	☆☆☆☆	☆☆☆	Critical small data
RAID 5	3	(N-1)/N	1 disk	☆☆☆☆	☆☆☆	General storage
RAID 6	4	(N-2)/N	2 disks	☆☆☆☆	☆☆	Large arrays
RAID 10	4	50%	1 per pair	☆☆☆☆ ☆	☆☆☆☆☆	Performance + redundancy

5.8 Software RAID vs Hardware RAID

Software RAID

Software RAID is implemented entirely by the operating system kernel. No special hardware is needed — the CPU handles all RAID calculations using the `mdadm` tool in Linux.

```
# Create software RAID 5 with mdadm
mdadm --create /dev/md0 --level=5 --raid-devices=3 \
    /dev/nvme0n2p1 /dev/nvme0n2p2 /dev/nvme0n2p3

# Monitor build
watch -n 2 cat /proc/mdstat

# Save configuration
mdadm --detail --scan >> /etc/mdadm.conf
```

Advantage	Disadvantage
No special hardware needed	CPU resources used for parity
Free — no additional cost	Slightly higher latency
Flexible — any disk type	OS must be running to access array
Easy to migrate to another system	No hardware write cache
Used in this lab	Not ideal for very large arrays

Hardware RAID

Hardware RAID uses a dedicated RAID controller card with its own processor, memory, and battery-backed write cache. The operating system sees only the logical volume — it has no knowledge of the underlying RAID configuration.

Common hardware RAID vendors:

- Broadcom/LSI MegaRAID
- HPE Smart Array
- Dell PERC (PowerEdge RAID Controller)
- Adaptec RAID

Advantage	Disadvantage
Offloads CPU completely	Expensive (\$200-\$2000+)
Battery-backed write cache	Controller failure = data inaccessible
Faster rebuild after failure	Vendor lock-in
OS-independent boot from RAID	Requires same controller model to recover
Better for large disk arrays	

Software vs Hardware RAID — When to Use Which

Scenario	Recommendation
Lab / learning environment	Software RAID (mdadm)
Small cluster, limited budget	Software RAID (mdadm)
Production HPC cluster	Hardware RAID
OS/boot volume protection	Hardware RAID (RAID 1)
Large storage arrays (12+ disks)	Hardware RAID
Lustre OST storage	Hardware RAID 6

Chapter 6: JBOD (Just a Bunch Of Disks)

6.1 Introduction

JBOD (Just a Bunch Of Disks) is a storage architecture where multiple physical disks are connected to a system but **not combined into a RAID array**. Each disk operates **independently** and is presented directly to the operating system.

JBOD is widely used in **high-performance computing (HPC)**, **large-scale storage systems**, and **parallel file systems** such as Lustre, Ceph, and GlusterFS, where storage management and redundancy are handled by software rather than hardware RAID.

In this configuration, the operating system detects each disk individually, allowing administrators to configure them according to application requirements.

6.2 Architecture

A typical JBOD setup consists of the following components:

1. **Disk Enclosure (JBOD Shelf)**
A hardware enclosure that holds multiple disks (commonly 12, 24, 36, or more drives).
2. **Host Bus Adapter (HBA)**
A controller installed in the server that connects the disks to the operating system via SAS or SATA interfaces.
3. **Connection Interface**
Usually **SAS cables** connecting the JBOD enclosure to the server HBA.
4. **Operating System**
The OS detects each disk separately and allows administrators to configure file systems or storage services on them.

Configuration:

JBOD Disk Shelf

|— Disk1

|— Disk2

|— SAS Connection

|

HBA Controller

|

Linux Storage Server

6.3 Characteristics of JBOD

Feature	Description
Disk Independence	Each disk operates independently
RAID	No RAID configuration
Redundancy	No built-in redundancy
Performance	Native disk performance
Failure Impact	Failure affects only that disk
Scalability	Easy to expand by adding disks

6.4 Advantages of JBOD

- Simple configuration and management
- Direct disk access without RAID controller overhead
- Cost-effective storage architecture
- Flexible disk usage
- Ideal for distributed and parallel file systems
- Easier disk replacement and expansion

6.5 Limitations of JBOD

- No inherent data redundancy
- No fault tolerance
- Manual disk management required
- Requires external backup or software-level redundancy

6.6 Configuration of JBOD –

After connecting the JBOD enclosure to the server, Linux will detect the disks automatically.

Check detected disks-

```
[root@hpc-master]# lsblk
```

Each disk can be partitioned using **fdisk** or **parted**.

```
[root@hpc-master]# fdisk /dev/sdb
```

- n (create new partition)
- p (primary)
- 1 (partition number)

- Enter
- Enter
- w (write changes)

Verify

```
[root@hpc-master]# lsblk
```

After partitioning, create a filesystem. Example using **XFS** (for large storage)

```
[root@hpc-master]# mkfs.xfs /dev/sdb1
```

Create mount directories:

```
[root@hpc-master]# mkdir /storage1
[root@hpc-master]# mkdir /storage2
[root@hpc-master]# mkdir /storage3
```

Mount disks:

```
[root@hpc-master]# mount /dev/sdb1 /storage1
[root@hpc-master]# mount /dev/sdc1 /storage2
[root@hpc-master]# mount /dev/sdd1 /storage3
```

Verify

```
[root@hpc-master]# df -h
```

To ensure disks mount automatically after reboot, update `/etc/fstab`

To find disk UUID:

```
[root@hpc-master]# blkid
```

```
vi /etc/fstab
```

Add entries:

```
UUID=e12a45b3-1234 /storage1 xfs defaults 0 0
UUID=ab34c5d6-5678 /storage2 xfs defaults 0 0
UUID=cd45e6f7-8912 /storage3 xfs defaults 0 0
```

Apply Changes:

```
[root@hpc-master]# mount -a
```

6.7 Monitoring JBOD Disks

Purpose	Command	Description
Check disk usage	<code>df -h</code>	Displays filesystem disk space usage in a human-readable format (GB/MB).
List block devices	<code>lsblk</code>	Shows all block devices such as disks and partitions along with their mount points.
Check disk health (SMART)	<code>smartctl -a /dev/sdb</code>	Displays SMART attributes and health information for the specified disk.
Monitor disk I/O activity	<code>iostat -x 2</code>	Provides extended disk I/O statistics and updates every 2 seconds.

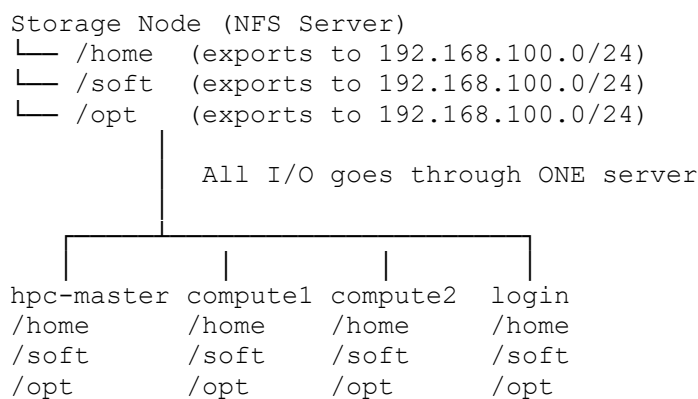
Chapter 7: File Systems — NFS vs Lustre

7.1 Why Shared Filesystems?

In an HPC cluster, jobs can run on any compute node. Without a shared filesystem, a job's output files would be written to the local disk of whichever node it ran on — making it impossible for the user to find their results consistently. Shared filesystems solve this by making the same directory visible on all nodes simultaneously.

7.2 NFS — Network File System

NFS (Network File System) is a distributed filesystem protocol developed by Sun Microsystems. A server exports directories over the network, and clients mount them as if they were local directories.



NFS Export Configuration:

```
# /etc/exports on storage node
/home    192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)
/soft    192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)
/opt     192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)

# Apply exports
exportfs -arv
```

NFS Client /etc/fstab:

```
192.168.100.12:/home /home nfs defaults,_netdev 0 0
192.168.100.12:/soft /soft nfs defaults,_netdev 0 0
192.168.100.12:/opt /opt nfs defaults,_netdev 0 0
```

Characteristics:

- Simple to configure and maintain
- Single server is a bottleneck and single point of failure
- Maximum throughput limited by one server's network bandwidth

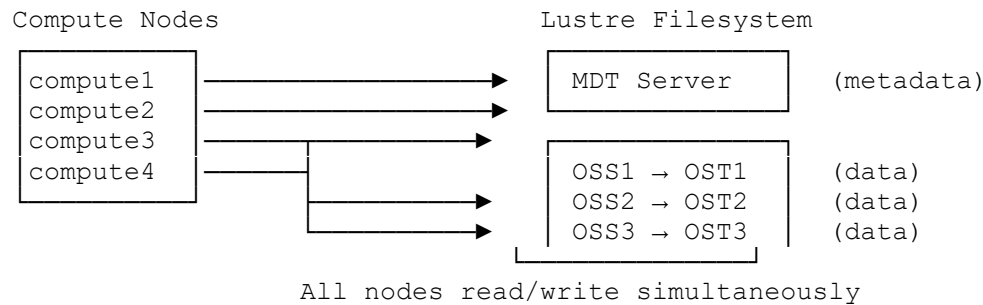
- Suitable for home directories, software installations, shared configuration files
- Not suitable for high-bandwidth parallel I/O from many nodes simultaneously

Typical NFS exports in HPC:

Export	Purpose
/home	User home directories
/soft	Shared scientific software (Ansys, Abaqus, etc.)
/opt	Optional system-wide software
/scratch	Temporary job working space

7.3 Lustre — Parallel File System (PFS)

Lustre is a high-performance parallel distributed filesystem designed specifically for HPC workloads. Unlike NFS where all traffic goes to one server, Lustre stripes files across multiple OSS servers — allowing all compute nodes to perform I/O simultaneously at full aggregate bandwidth.



File Striping in Lustre:

A single large file can be striped across multiple OSTs, so reading/writing it uses the bandwidth of all OSTs simultaneously:

```
# Set stripe count for a directory (stripe across 4 OSTs)
lfs setstripe -c 4 /scratch/myproject/

# Check stripe settings
lfs getstripe /scratch/myproject/largefile.dat
```

Characteristics:

- Scales to petabytes of storage and thousands of nodes
 - Aggregate bandwidth scales linearly with number of OSSes
 - Used by the majority of TOP500 supercomputers
 - Significantly more complex to set up and maintain than NFS
 - Supports file striping across multiple disks for parallel I/O
 - Requires dedicated MDT and OSS servers
-

7.4 NFS vs Lustre — Side by Side Comparison

Feature	NFS	Lustre
Setup Complexity	Simple	Complex
Scalability	Limited (single server)	Extremely high
Aggregate Bandwidth	One server (e.g., 5 GB/s)	Sum of all OSSes (e.g., 500 GB/s)
Latency	Higher	Lower
Parallel I/O	No	Yes
Metadata Performance	Moderate	High (dedicated MDT)
Fault Tolerance	Basic	High (replicated MDT available)
Administration	Simple	Complex
Cost	Low	High
Typical HPC Role	/home, /soft, /opt	/scratch, /work, simulation data

Chapter 8: HPC Software Stack

8.1 Chrony — Time Synchronization

Chrony is an implementation of NTP (Network Time Protocol) that keeps all node clocks synchronized. Time synchronization is critically important in HPC clusters for several reasons:

Reason	Consequence of Poor Sync
MUNGE authentication	MUNGE tokens expire — authentication fails if clocks differ by more than 5 minutes
Kerberos (IPA)	Kerberos requires clocks within 5 minutes — jobs fail to authenticate
Log correlation	Comparing logs across nodes requires synchronized timestamps
Job accounting	Inaccurate job duration and resource usage records
Filesystem consistency	NFS and Lustre rely on consistent timestamps

Configuration in this lab:

- hpc-master: local stratum 10 server (no upstream NTP — lab environment, UDP 123 blocked by VMware)
- All other nodes: sync from hpc-master (stratum 11)

```
# On hpc-master (NTP server)
cat > /etc/chrony.conf << EOF
local stratum 10
allow 192.168.100.0/24
driftfile /var/lib/chrony/drift
makestep 1.0 3
EOF

# On compute nodes (NTP clients)
cat > /etc/chrony.conf << EOF
server 192.168.100.1 iburst
makestep 1.0 3
driftfile /var/lib/chrony/drift
EOF

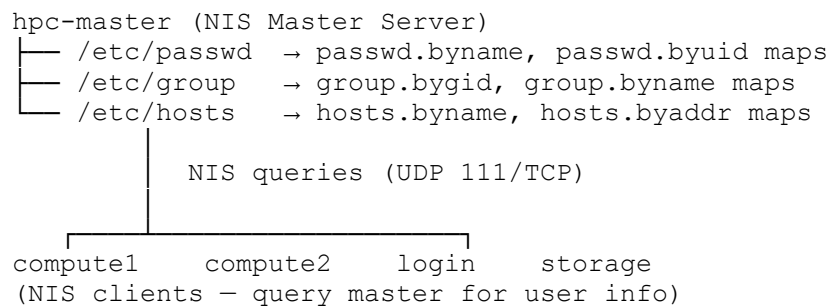
# Verify synchronization
chronyc tracking
chronyc sources
```

8.2 NIS — Network Information Service

NIS (Network Information Service) provides centralized management of system configuration data — usernames, passwords, group memberships, hostnames — across multiple nodes in a network. It was the traditional method for user management in UNIX/Linux clusters before LDAP and IPA.

Why NIS matters in HPC: When a job runs on compute1 as user "amogh" (UID 1500), the same UID must exist on compute1 with the same permissions. Without centralized user management, UIDs could differ across nodes, causing permission errors and job failures.

Architecture:



Key NIS commands:

```
# Server
/usr/lib64/yp/ypinit -m          # Initialize NIS domain
cd /var/yp && make                # Rebuild NIS maps after changes
ypserv                          # NIS server daemon

# Client
ypwhich                          # Which NIS server am I using?
ypcat passwd                    # Show NIS password map
getent passwd <username>        # Resolve user via NIS
```

NIS transmits passwords in plaintext and has known security vulnerabilities. It is being replaced by FreeIPA/LDAP in modern HPC environments. Use NIS only in isolated, trusted networks.

8.3 FreeIPA — Identity Management

FreeIPA (Free Identity, Policy and Audit) is a comprehensive, integrated identity and authentication solution for Linux environments. It is the modern, secure replacement for NIS in enterprise HPC clusters. It combines LDAP directory services, Kerberos authentication, DNS, and certificate management into a single system.

Components:

Component	Technology	Purpose
Identity	389 Directory Server (LDAP)	Stores users, groups, hosts, policies
Authentication	MIT Kerberos KDC	Secure ticket-based single sign-on
DNS	BIND	Name resolution for the IPA domain
Certificate Authority	Dogtag PKI	Issues SSL/TLS certificates
Policy Enforcement	SSSD	Applies access control on clients
Web Interface	Apache	Browser-based administration

Advantages over NIS:

Feature	NIS	FreeIPA
Password transmission	Plaintext	Encrypted (Kerberos)
Authentication	Simple	Kerberos SSO
Access control	Basic	Fine-grained HBAC
Multi-factor auth	No	Yes
Web UI	No	Yes
Certificate management	No	Yes
Modern HPC use	Legacy	Standard

Key commands:

```
# Get Kerberos ticket
kinit admin

# User management
ipa user-add <username> --first=<First> --last=<Last> --password
ipa user-find
ipa user-show <username>
ipa user-disable <username>

# Group management
ipa group-add <groupname>
ipa group-add-member <groupname> --users=<username>

# Host management
ipa host-find
```

8.4 xCAT — Extreme Cluster Administration Toolkit

xCAT is an open-source cluster management toolkit originally developed by IBM. It is one of the most widely used tools for deploying and managing large-scale Linux HPC clusters from bare metal.

Key capabilities:

Capability	Description
PXE Provisioning	Boot nodes over network and auto-install OS
Node Discovery	Automatically discover new hardware
Parallel Execution	Run commands on hundreds of nodes simultaneously
Hardware Control	Power management, console access via IPMI
Infrastructure Services	Manages DHCP, DNS, TFTP for cluster
Image Management	Create, customize, and deploy OS images

Key xCAT commands:

<code>lsxcatd -a</code>	<code># Check xCAT service status</code>
<code>lsdef -t node</code>	<code># List all defined nodes</code>
<code>chdef <node> ip=<IP> mac=<MAC></code>	<code># Define node properties</code>
<code>nodeset <node> osimage=<image></code>	<code># Prepare node for PXE boot</code>
<code>rpowers <node> boot</code>	<code># Power cycle node</code>
<code>xdsh <nodes> <command></code>	<code># Run command on multiple nodes</code>
<code>xdcp <nodes> <source> <dest></code>	<code># Copy files to multiple nodes</code>
<code>tabdump networks</code>	<code># Show network configuration</code>

8.5 HPCM — HPE HPC Cluster Manager

HPCM (HPC Cluster Manager) is HPE's enterprise cluster management solution, designed specifically for HPE hardware environments. It provides similar functionality to xCAT but with tighter integration with HPE hardware management tools.

Key features:

- Automated cluster deployment and lifecycle management
- Native integration with HPE iLO (Integrated Lights-Out) for hardware management
- Node health monitoring and alerting
- Rolling OS updates without cluster downtime
- Pre-built integration with Slurm and PBS schedulers
- Support for RHEL, Rocky Linux, SLES

Typical use: HPCM is commonly deployed in HPE Apollo systems and HPE Cray EX supercomputers. It is the cluster management tool of choice in many commercial HPC environments running HPE hardware.

Chapter 9: Job Schedulers

9.1 Why Job Schedulers?

Without a job scheduler, users would need to manually decide which compute nodes to use, manually SSH into them, manually run their programs, and manually ensure they don't conflict with other users. This is completely impractical with tens or hundreds of simultaneous users.

A job scheduler solves this by:

- Accepting job submissions with resource requirements
- Maintaining a queue of pending jobs
- Automatically placing jobs on suitable nodes when resources are available
- Enforcing fair use policies and resource limits
- Tracking job history and usage accounting

9.2 Slurm — Simple Linux Utility for Resource Management

Slurm is the most widely used open-source HPC workload manager, used by the majority of the world's TOP500 supercomputers. It was originally developed at Lawrence Livermore National Laboratory and is now maintained by SchedMD.

Architecture:

```
hpc-master
├─ slurmctld    ← Central controller (job management, scheduling)
├─ slurmdbd    ← Database daemon (job accounting – optional)
└─ MUNGE       ← Authentication

computel, compute2
├─ slurmd      ← Worker daemon (executes jobs)
└─ MUNGE       ← Authentication (same key as master)
```

MUNGE Authentication: MUNGE (MUNGE Uid 'N' Gid Emporium) creates cryptographically signed credentials that Slurm uses to authenticate communication between nodes. All nodes must share an identical MUNGE key.

```
# Generate MUNGE key
dd if=/dev/urandom of=/etc/munge/munge.key bs=1 count=1024
chown munge:munge /etc/munge/munge.key
chmod 400 /etc/munge/munge.key
chattr +i /etc/munge/munge.key # Prevent auto-regeneration

# Copy to all nodes (key must be identical everywhere)
scp /etc/munge/munge.key root@computel:/etc/munge/
scp /etc/munge/munge.key root@compute2:/etc/munge/
```



```
# Verify keys are identical
md5sum /etc/munge/munge.key
ssh root@compute1 "md5sum /etc/munge/munge.key"
```

slurm.conf — Key Parameters:

```
ClusterName=hpccluster
ControlMachine=hpc-master
ControlAddr=192.168.100.1
AuthType=auth/munge
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
SlurmUser=slurm

# Node definitions
NodeName=compute1 NodeAddr=192.168.100.13 CPUs=2 RealMemory=2900
State=UNKNOWN
NodeName=compute2 NodeAddr=192.168.100.14 CPUs=2 RealMemory=2900
State=UNKNOWN

# Partition definitions
PartitionName=normal Nodes=compute1,compute2 Default=YES MaxTime=INFINITE
State=UP
```

Key Slurm commands:

Command	Purpose
sbatch job.sh	Submit batch job
squeue	View job queue
sinfo	View partition and node status
scancel <jobid>	Cancel a job
sacct -j <jobid>	Job accounting history
srun --pty bash	Launch interactive job
scontrol show job <id>	Detailed job information
scontrol show node <name>	Detailed node information

Sample Slurm job script:

```
#!/bin/bash
#SBATCH --job-name=MySimulation
#SBATCH --partition=normal
#SBATCH --nodes=2
#SBATCH --ntasks=16
#SBATCH --cpus-per-task=1
#SBATCH --mem=4G
#SBATCH --time=02:00:00
#SBATCH --output=job_%j.out
#SBATCH --error=job_%j.err

module load mpi/openmpi-x86_64
mpirun -np 16 ./my_simulation input.dat
```

SBATCH Directives:

Directive	Example	Meaning
--job-name	--job-name=CFD_Run	Job name
--partition	--partition=normal	Queue/partition
--nodes	--nodes=4	Number of nodes
--ntasks	--ntasks=32	Total MPI tasks
--ntasks-per-node	--ntasks-per-node=8	Tasks per node
--cpus-per-task	--cpus-per-task=4	OpenMP threads
--mem	--mem=64G	Memory per node
--time	--time=24:00:00	Max walltime
--gres	--gres=gpu:2	GPU resources
--output	--output=job_%j.out	Stdout file

9.3 OpenPBS — Portable Batch System

OpenPBS is another widely used open-source job scheduler, originally developed by NASA for managing jobs on the Space Shuttle project. It is now maintained by Altair Engineering as an open-source project.

Architecture:

```
hpc-master
├─ pbs_server ← Accepts and manages jobs
├─ pbs_sched  ← Scheduler (decides job placement)
└─ pbs_comm   ← Communication router

compute1, compute2
└─ pbs_mom    ← Machine Oriented Mini-server (executes jobs)

login
└─ PBS client ← Job submission tools only (qsub, qstat)
```

Key PBS commands:

Command	Purpose
qsub job.sh	Submit batch job
qstat -a	View job queue
pbsnodes -a	View node status
qdel <jobid>	Cancel a job
qstat -B	Server status
qstat -Q	Queue status
qmgr	Administrator management interface

Sample PBS job script:

```
#!/bin/bash
#PBS -N MySimulation
#PBS -l select=2:ncpus=8:mem=32gb
#PBS -l walltime=02:00:00
#PBS -q workq

cd $PBS_O_WORKDIR
mpirun -np 16 ./my_simulation input.dat
```

9.4 Slurm vs OpenPBS — Comparison

Feature	Slurm	OpenPBS
Job submission	sbatch	qsub
Job monitoring	squeue	qstat -a
Cancel job	scancel	qdel
Cluster status	sinfo	pbsnodes -a
Node daemon	slurmd	pbs_mom
Controller	slurmctld	pbs_server
Scheduler	Built-in	pbs_sched
Authentication	MUNGE	MUNGE
Config file	slurm.conf	pbs.conf
Script directives	#SBATCH	#PBS
TOP500 usage	~70%	~20%
Commercial support	SchedMD	Altair
GPU scheduling	Native	Supported
Best for	Large clusters	Commercial HPC

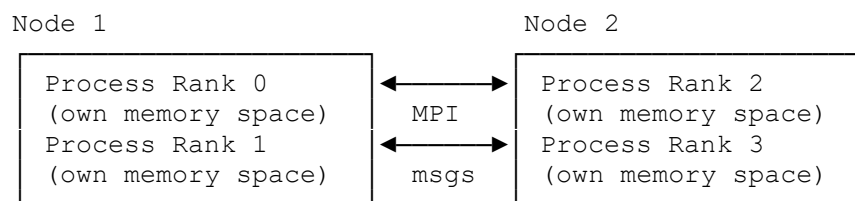
Chapter 10: MPI and Parallel Programming

10.1 What is MPI?

MPI (Message Passing Interface) is a standardized, portable message-passing programming model designed for parallel computing on distributed memory systems. It is the dominant programming model for writing scientific applications that run across multiple nodes of an HPC cluster.

MPI allows a program to launch as many simultaneous processes across many nodes, with each process having its own private memory. Processes communicate by explicitly sending and receiving messages — unlike shared memory programming (OpenMP) where threads share the same memory space.

The MPI execution model:



Key MPI concepts:

Concept	Description
Rank	Unique integer ID of each process (0 to N-1)
Size	Total number of processes in the job
Communicator	Group of processes that can communicate (MPI_COMM_WORLD = all)
Point-to-point	Communication between two specific processes (MPI_Send/MPI_Recv)
Collective	Communication involving all processes simultaneously

Common MPI collective operations:

Operation	Description	Use Case
MPI_Bcast	Broadcast data from rank 0 to all	Distribute initial parameters
MPI_Scatter	Distribute array segments to all	Distribute work chunks
MPI_Gather	Collect results from all to rank 0	Collect partial results
MPI_Reduce	Combine values from all (sum, max, etc.)	Global reduction operations
MPI_Allreduce	Reduce and distribute result to all	Most common in simulations
MPI_Barrier	Synchronize all processes	Ensure all reach the same point
MPI_Alltoall	Every process sends to every other	FFT operations

Simple MPI example program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get this process's rank and total number of processes
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello from process %d of %d on %s\n",
           rank, size, getenv("HOSTNAME"));

    // Only rank 0 prints the total
    if (rank == 0) {
        printf("Total processes: %d\n", size);
    }

    // Synchronize all processes
    MPI_Barrier(MPI_COMM_WORLD);

    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Compile and run:

```
# Compile
mpicc -o hello_mpi hello_mpi.c

# Run on 8 processes across 2 nodes
mpirun -np 8 -hostfile hosts ./hello_mpi

# Run via Slurm
sbatch --nodes=2 --ntasks=8 --wrap="mpirun ./hello_mpi"
```

10.2 OpenMPI

OpenMPI is the most widely used open-source implementation of the MPI standard. It is a community-driven project maintained by a consortium of academic, research, and industry partners.

Installation:

```
# Install via package manager
dnf install openmpi openmpi-devel -y

# Load environment
module load mpi/openmpi-x86_64

# Verify
mpirun --version
mpicc --version
```

Key features:

- Supports InfiniBand, Ethernet, and shared memory transports
- RDMA (Remote Direct Memory Access) support for InfiniBand
- Integration with Slurm via PMIx process management
- Supports MPI-3.1 standard including non-blocking collectives
- Fault tolerance capabilities
- GPU-aware MPI (can transfer data directly between GPUs)

Running MPI with Slurm:

```
#!/bin/bash
#SBATCH --job-name=MPI_Simulation
#SBATCH --nodes=4
#SBATCH --ntasks=64
#SBATCH --ntasks-per-node=16
#SBATCH --time=04:00:00
#SBATCH --output=mpi_job_%j.out

module load mpi/openmpi-x86_64

# srun is preferred over mpirun when using Slurm
srun ./my_mpi_program input.dat
```

MPI Implementations Comparison:

Implementation	Maintained By	Optimized For	Notes
OpenMPI	Open source consortium	General	Most widely used
MPICH	Argonne National Lab	Portability	Reference implementation
Intel MPI	Intel	Intel Xeon / Omni-Path	Best for Intel hardware
MVAPICH2	Ohio State University	InfiniBand RDMA	Best IB performance
HPE MPI	HPE	HPE / Cray systems	Cray supercomputers
IBM Spectrum MPI	IBM	IBM Power systems	IBM clusters

10.3 MPI + OpenMP Hybrid Programming

Many modern HPC applications use a hybrid approach — MPI between nodes and OpenMP threads within each node. This is often more efficient than pure MPI because:

- Reduces MPI communication overhead (fewer processes per node)
- OpenMP threads share memory within a node (no message passing needed)
- Better utilization of multi-core CPUs with large memory

```
#!/bin/bash
#SBATCH --nodes=4
#SBATCH --ntasks=4          # 1 MPI process per node
#SBATCH --cpus-per-task=16   # 16 OpenMP threads per MPI process
#SBATCH --time=02:00:00

module load mpi/openmpi-x86_64
export OMP_NUM_THREADS=16

# 4 MPI processes × 16 OpenMP threads = 64 total cores
mpirun -np 4 ./hybrid_simulation
```

Chapter 11: Benchmarking HPC Cluster

11.1 Introduction

Benchmarking an HPC cluster is the process of measuring the **performance, efficiency, and scalability** of compute, network, and storage resources. It helps administrators validate system performance after deployment and ensures that the cluster performs as expected for scientific or enterprise workloads.

Benchmarking is commonly performed:

- After **cluster installation**
- After **hardware upgrades**
- For **performance validation**
- For **TOP500/HPC procurement evaluation**
- For **performance tuning**

Benchmarking typically evaluates three major subsystems:

- **Compute performance**
- **Network latency and bandwidth**
- **Storage throughput and I/O performance**

11.2 Benchmark Categories

Benchmark Type	Purpose	Common Tools
CPU / Compute Benchmark	Measure floating-point performance	HPL, HPCG, Linpack
Memory Benchmark	Measure memory bandwidth and latency	STREAM
Network Benchmark	Measure interconnect latency and bandwidth	OSU Micro Benchmarks
Storage Benchmark	Measure I/O throughput and IOPS	IOzone, FIO
GPU Benchmark	Measure GPU computational performance	CUDA samples, HPL-AI

11.3 Common HPC Benchmark Tools

Tool	Purpose
HPL (High Performance Linpack)	Measures floating-point compute performance
HPCG	Evaluates memory and communication performance
STREAM	Measures memory bandwidth
OSU Micro Benchmarks (OMB)	Tests MPI latency and bandwidth
IOzone / FIO	Storage performance benchmarking

11.4 Prerequisites for Benchmarking

Before running benchmarks, ensure:

- Cluster nodes are **reachable and operational**
- MPI stack installed (OpenMPI / Intel MPI)
- Passwordless SSH configured
- Job scheduler configured (PBS, Slurm, etc.)
- Shared storage accessible
- Benchmark tools installed

Basic node connectivity check:

```
[root@hpc-master]# pdsh -w compute[01-02]
```

Check MPI installation:

```
[root@hpc-master]# mpirun --version
```

11.5 CPU Benchmark – HPL (High Performance Linpack)

HPL is the most widely used benchmark in HPC systems and is used for **TOP500 rankings**.

Step 1 – Download HPL

```
[root@hpc-master]# wget http://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
```

Extract:

```
[root@hpc-master]# tar -xvf hpl-2.3.tar.gz
[root@hpc-master]# cd hpl-2.3
```

Step 2 – Configure HPL

Copy a sample configuration:

```
[root@hpc-master]# cp setup/Make.Linux_PII_FBLAS Make.arch
```

Edit configuration:

```
vi Make.arch
```

Specify:

- MPI library
 - BLAS library
 - compiler flags
-

Step 3 – Compile HPL

```
[root@hpc-master]# make arch=Linux_PII_FBLAS
```

Binary will be created in:

```
[root@hpc-master]# bin/Linux_PII_FBLAS/xhpl
```

11.6 Configure HPL.dat

Edit benchmark configuration:

```
[root@hpc-master]# vi HPL.dat
```

Important parameters:

Parameter	Description
Problem Size (N)	Matrix size
Block Size (NB)	Computation block
Process Grid (P x Q)	MPI process layout

Example:

```
N = 100000
NB = 256
P = 4
Q = 4
```

11.7 Run HPL Benchmark

Run using MPI:

```
[root@hpc-master]# mpirun -np 16 ./xhpl
```

Example output:

```
WR00L2L4 100000 256 4 4 1200.34 5.6e+03
```

Performance is measured in **GFLOPS or TFLOPS**.

11.8 Memory Benchmark – STREAM

STREAM measures **memory bandwidth performance**.

Step 1 – Download STREAM

```
[root@hpc-master]# wget  
https://www.cs.virginia.edu/stream/FTP/Code/stream.c
```

Step 2 – Compile STREAM

```
[root@hpc-master]# gcc -O3 -fopenmp stream.c -o stream
```

Step 3 – Run Benchmark

```
[root@hpc-master]# ./stream
```

Example output:

```
Copy: 45000 MB/s  
Scale: 44000 MB/s  
Add: 42000 MB/s  
Triad: 43000 MB/s
```

This indicates **memory bandwidth in MB/s**.

11.9 Network Benchmark – OSU Micro Benchmarks

OSU benchmarks measure **MPI latency and bandwidth**.

Step 1 – Download

```
[root@hpc-master]# wget https://mvapich.cse.ohio-  
state.edu/download/mvapich/osu-micro-benchmarks-7.5.tar.gz
```

Step 2 – Install the Package

```
[root@hpc-master]# tar -xvf osu-micro-benchmarks-7.5.tar.gz  
[root@hpc-master]# cd osu-micro-benchmarks-7.5
```

Configure with MPI –

```
[root@hpc-master]# ./configure CC=mpicc CXX=mpicxx
```

Compile –

```
[root@hpc-master]# make
```

Step 3 – Run Latency Test

```
[root@hpc-master]# mpirun -np 2 -host node01,node02  
./c/mpi/pt2pt/standard/osu_latency
```

Example output:

#	Size	Latency(us)
0		1.2
8		1.3
64		1.4
512		1.6

Step 4 – Run Bandwidth Test

```
[root@hpc-master]# mpirun -np 2 -host node01,node02  
./c/mpi/pt2pt/standard/osu_bw
```

Example output:

#	Size	Bandwidth (MB/s)
1		1500
1024		9000
8192		12000

11.10 Storage Benchmark – IOzone

IOzone tests **filesystem performance**.

Install IOzone

```
[root@hpc-master]# dnf install iozone -y
```

Run Benchmark

```
[root@hpc-master]# iozone -a -i 0 -i 1 -i 2 -f /storage/testfile
```

This tests:

- write
- read
- random read/write

Example output:

```
Write:      1500 MB/s
Read:       1800 MB/s
Random Read:  900 MB/s
Random Write: 800 MB/s
```

11.11 Monitoring During Benchmark

Useful commands during benchmarking:

Purpose	Command	Description
CPU usage	top	Monitor CPU load
Node resource usage	htop	Advanced CPU and memory monitoring
Disk I/O monitoring	iostat -x 2	Disk throughput statistics
Network monitoring	iftop	Monitor network traffic
MPI process monitoring	ps -ef grep mpi	Check MPI jobs

11.12 Benchmark Result Analysis

Key metrics to analyze:

Metric	Meaning
GFLOPS / TFLOPS	Compute performance
Memory bandwidth	Memory subsystem efficiency
Network latency	MPI communication delay
Network bandwidth	Inter-node data transfer rate
IOPS / Throughput	Storage performance

Benchmarking is an essential process in HPC cluster deployment and maintenance. It validates system performance, identifies potential bottlenecks, and ensures that compute, network, and storage resources operate efficiently. By using industry-standard benchmarking tools such as **HPL**, **STREAM**, **OSU Micro Benchmarks**, and **IOzone**, administrators can comprehensively evaluate cluster performance and optimize it for production workloads.

Chapter 12: HPC Cluster Setup — Step by Step

Step 1 — Bill of Materials (BoM) Verification

Before beginning physical installation, carefully verify all hardware components against the Bill of Materials document provided by the vendor.

Verification checklist:

Hardware Nodes:

- ☐ Count all servers — head nodes, login, compute, storage
- ☐ Verify server model numbers against BoM
- ☐ Verify CPU model and count per server
- ☐ Verify RAM capacity and module configuration
- ☐ Verify disk type, size, and quantity per server
- ☐ Verify NIC cards (model, port count, speed)
- ☐ Verify InfiniBand HCA cards (model, speed)
- ☐ Verify GPU cards (if applicable)

Networking:

- ☐ Verify Ethernet switch model and port count
- ☐ Verify InfiniBand switch model and port count
- ☐ Count all network cables (type and length)
- ☐ Count all InfiniBand cables (QSFP, HDR)
- ☐ Verify transceivers/DAC cables for 25/100 GbE

Power & Physical:

- ☐ Verify PDU model and outlet count
- ☐ Verify power cables (type, length, connector type)
- ☐ Verify rack units (U) and rack dimensions
- ☐ Verify rail kits for all servers
- ☐ Verify cable management accessories

Storage:

- ☐ Verify disk shelf model and capacity
- ☐ Verify HBA (Host Bus Adapter) cards
- ☐ Verify SAS/SATA cables for disk shelves
- ☐ Verify any additional SSDs or NVMe drives

Document any missing, damaged, or incorrect items immediately and raise a vendor support ticket before proceeding.

Step 2 — Rack Planning

Plan the physical equipment layout before installing anything. A well-planned rack layout improves airflow, cable management, and ease of maintenance.

Airflow considerations:

- HPC servers use front-to-back airflow (cold air in front, hot air out rear)
- Data center must have cold aisle / hot aisle separation
- Never place servers facing opposite directions in the same rack
- Blanking panels must fill all empty rack units to prevent hot air recirculation

Typical rack layout (top to bottom):

```
U1   - U2   : Patch panels (network)
U3   - U4   : 1 GbE Management Switch
U5   - U6   : 25 GbE Service Switch
U7   - U8   : InfiniBand HDR Switch
U9   - U10  : Head Node (hpc-master)
U11  - U12  : Login Node
U13  - U32  : Compute Nodes (10 × 2U each)
U33  - U36  : Storage Nodes
U37  - U38  : Storage Disk Shelf
U39  - U40  : PDU (or vertical PDU at rear)
```

Power planning:

- Calculate total power draw per rack (watts)
 - Verify PDU amperage capacity vs total server power
 - Plan A/B power paths for redundancy
 - Verify facility breaker ratings
-

Step 3 — Rack Stacking and Mounting

Physical installation of equipment into racks:

Procedure:

1. Install rack rails for each server following the manufacturer's rail kit instructions
 2. Always use two people when installing servers — most 2U servers weigh 15-25 kg
 3. Use a server lift for heavy storage arrays (40+ kg)
 4. Mount heaviest equipment at the bottom of the rack (storage arrays, UPS)
 5. Slide servers onto rails and secure with mounting screws
 6. Install blanking panels in all empty rack units immediately
 7. Mount switches at planned positions (leave space for cable routing)
 8. Mount PDUs — vertical PDUs attach to the rear vertical rails
 9. Install cable management arms (CMAs) on servers
 10. Affix asset tags and hostname labels to all equipment
-

Step 4 — Power Cabling

Connect power distribution from PDUs to all equipment:

Procedure:

1. With PDUs powered OFF, connect all power cables
2. Connect each server's primary PSU to PDU-A
3. Connect each server's secondary/redundant PSU to PDU-B
4. Verify PDU-A and PDU-B are on separate facility circuits
5. Label all power cables at both ends (server name + PSU number)
6. Route cables through cable management panels
7. Verify no cable creates a trip hazard or blocks airflow
8. Have facility electrician verify input power before energizing

Never power on equipment until all cabling is complete and verified. Partial cabling can result in servers booting without redundant power.

Step 5 — Network Cabling

Connect all network cables following the rack diagram:

Network cabling order:

1. Cable IPMI/BMC ports of all servers to management switch
2. Cable data NICs (25 GbE) of all servers to service switch
3. Cable InfiniBand HCAs of compute/storage nodes to IB switch
4. Cable switch-to-switch uplinks (management → service → IB → core)
5. Cable head node to all switches (management + service + IB)
6. Cable external uplink from cluster to datacenter network

Cable labeling convention:

Label format: <source_device>:<port> → <dest_device>:<port>

Example: compute1:ens160 → eth-sw1:Gi0/1

Example: compute1:mlx5_0 → ib-sw1:port12

Color coding recommendation:

Blue cables : Management / IPMI network (1 GbE)

Green cables : Service / data network (25 GbE)

Yellow cables : InfiniBand (HDR)

Red cables : Power (never confuse with data)

Step 6 — Initial Power-On

First power-on sequence:

Procedure:

1. Power on PDUs — verify input voltage readings are correct
 2. Power on network switches first — verify all LEDs are green
 3. Power on head node — monitor POST (Power-On Self-Test) via console
 4. Verify no error LEDs (amber = warning, red = fault)
 5. Check IPMI/BMC is accessible from management laptop
 6. Power on remaining nodes one at a time
 7. Verify fan operation and check for thermal alarms
 8. Check that all NICs and HBAs are detected in IPMI
-

Step 7 — BIOS Configuration and Firmware Update

Configure BIOS and update all firmware before OS installation:

Critical BIOS settings for HPC:

Performance Settings:

CPU Power Management	: Maximum Performance
Turbo Boost	: Enabled
Hyper-Threading	: Enabled (most workloads) / Disabled (some HPC)
NUMA	: Enabled
Memory Frequency	: Maximum supported

Boot Settings:

Boot Order	: Network (PXE) first, then local disk
Secure Boot	: Disabled (required for PXE provisioning)

Virtualization:

VT-x / AMD-V	: Enabled
SR-IOV	: Enabled (for InfiniBand passthrough)

Power:

Power Profile	: Maximum Performance (not OS control)
C-States	: Disabled (for low-latency MPI)

Firmware update order:

1. BMC/iLO/iDRAC firmware ← Update first
2. BIOS firmware
3. NIC firmware
4. HBA (SAS controller) firmware
5. InfiniBand HCA firmware
6. Disk firmware (HDDs/SSDs)
7. Switch firmware (IB and Ethernet)

Always update firmware BEFORE OS installation. Firmware updates sometimes require a full system reboot and can change hardware behavior.

Step 8 — Network and IP Planning

Document all IP addresses before beginning OS installation:

IP Address List Example:

Node	IPMI/BMC – Ilo/IDRAC IP	Management (1G)	Cluster (25G) – Internal IP	InfiniBand
hpc-master	10.0.0.1	172.16.0.1	192.168.100.1	10.10.0.1
login	10.0.0.2	172.16.0.2	192.168.100.11	—
compute1	10.0.0.3	172.16.0.3	192.168.100.13	10.10.0.13
compute2	10.0.0.4	172.16.0.4	192.168.100.14	10.10.0.14
storage	10.0.0.5	172.16.0.5	192.168.100.12	10.10.0.12

DNS naming convention:

```
Domain: hpc.local
hpc-master.hpc.local  192.168.100.1
compute1.hpc.local    192.168.100.13
compute2.hpc.local    192.168.100.14
login.hpc.local        192.168.100.11
storage.hpc.local      192.168.100.12
```

Step 9 — OS Installation

Head node — Manual installation from ISO:

1. Boot from RHEL 9.4 ISO (USB or virtual media via IPMI)
2. Select "Minimal Install" (no GUI for servers)
3. Configure disk: LVM with recommended partitioning
4. Set hostname: `hpc-master.hpc.local`
5. Configure network: static IP on management interface
6. Set root password and create admin user
7. Complete installation and reboot

Recommended disk partitioning:

/boot	1 GB	XFS	(standard partition)
/boot/efi	600 MB	FAT32	(UEFI systems)
/	50 GB	XFS	(LVM)
/var	20 GB	XFS	(LVM — for logs)
/tmp	10 GB	XFS	(LVM)
swap	32 GB	swap	(LVM — or RAM size up to 64 GB)

Compute/login/storage nodes — Automated via xCAT PXE:

```
# On head node after xCAT is configured
nodeadd compute1 groups=compute,all
chdef compute1 ip=192.168.100.13 mac=<MAC_ADDRESS> \
  netboot=xnba installnic=ens160

nodeset compute1 osimage=rhel9.4-x86_64-install-compute
rpower compute1 boot
```

Post-installation on all nodes:

```
# Disable firewall
systemctl disable --now firewalld

# Disable SELinux
sed -i 's/^SELINUX=.*SELINUX=disabled/' /etc/selinux/config
setenforce 0

# Update all packages
dnf update -y

# Install essential tools
dnf install -y vim wget curl net-tools bind-utils \
  nfs-utils chrony munge openssh-server

# Configure /etc/hosts
cat >> /etc/hosts << EOF
192.168.100.1   hpc-master.hpc.local hpc-master
192.168.100.11 login.hpc.local      login
192.168.100.13 compute1.hpc.local   compute1
192.168.100.14 compute2.hpc.local   compute2
192.168.100.12 storage.hpc.local    storage
EOF
```

Step 10 — NFS Configuration

On storage node (NFS server):

```
dnf install nfs-utils -y
mkdir -p /home /soft /opt

cat > /etc/exports << EOF
/home    192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)
/soft    192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)
/opt     192.168.100.0/24(rw,sync,no_root_squash,no_subtree_check)
EOF

systemctl enable --now nfs-server rpcbind
exportfs -arv
showmount -e localhost
```

On all client nodes:

```
dnf install nfs-utils -y
mkdir -p /home /soft /opt

# Add to /etc/fstab
cat >> /etc/fstab << EOF
192.168.100.12:/home /home nfs defaults,_netdev 0 0
192.168.100.12:/soft /soft nfs defaults,_netdev 0 0
192.168.100.12:/opt /opt nfs defaults,_netdev 0 0
EOF

systemctl enable rpcbind
mount -a
df -h | grep 192.168.100.12
```

Step 11 — NTP Configuration

```
# On head node (NTP server)
dnf install chrony -y
cat > /etc/chrony.conf << EOF
server 0.rhel.pool.ntp.org iburst
server 1.rhel.pool.ntp.org iburst
local stratum 10
allow 192.168.100.0/24
driftfile /var/lib/chrony/drift
makestep 1.0 3
EOF
systemctl enable --now chronyd

# On all client nodes
dnf install chrony -y
cat > /etc/chrony.conf << EOF
server 192.168.100.1 iburst
makestep 1.0 3
driftfile /var/lib/chrony/drift
EOF
systemctl enable --now chronyd
chronyc sources
```

Step 12 — IPA Configuration

```
# On head node – install IPA server
dnf module enable idm:DL1 -y
dnf install ipa-server ipa-server-dns -y

ipa-server-install \
  --domain=hpc.local \
  --realm=HPC.LOCAL \
  --hostname=hpc-master.hpc.local \
  --ip-address=192.168.100.1 \
  --no-ntp \
  --setup-dns \
  --forwarder=8.8.8.8

# Create cluster users
kinit admin
ipa user-add hpcuser1 --first=HPC --last=User1 --password
ipa user-add hpcuser2 --first=HPC --last=User2 --password

# On all client nodes – install IPA client
dnf install freeipa-client -y
ipa-client-install --mkhomedir \
  --server=hpc-master.hpc.local \
  --domain=hpc.local \
  --realm=HPC.LOCAL \
  --hostname=$(hostname -f) \
  -p admin -w <password> \
  --unattended
```

Step 13 — OpenPBS Installation

```
# On head node
dnf install openpbs-server -y
/etc/init.d/pbs start
systemctl enable pbs

qmgr -c "set server flatuid=true"
qmgr -c "set server scheduling=true"
qmgr -c "set server job_history_enable=true"
qmgr -c "set queue workq queue_type=execution"
qmgr -c "set queue workq started=true"
qmgr -c "set queue workq enabled=true"
qmgr -c "set server default_queue=workq"
qmgr -c "create node compute1"
qmgr -c "create node compute2"
qmgr -c "set node compute1 resources_available.ncpus=32"
qmgr -c "set node compute2 resources_available.ncpus=32"

# On compute nodes
dnf install openpbs-execution -y
# Set PBS_SERVER=hpc-master.hpc.local in /etc/pbs.conf
/opt/pbs/sbin/pbs_mom

# On login node
dnf install openpbs-client -y
# Verify
qstat -B
pbsnodes -a
```

Step 14 — Application Installation

Ansys (CFD and FEA)

```
# Mount installer
mount -o loop ANSYS_<version>_LINUX64.iso /mnt/iso

# Run silent install
/mnt/iso/INSTALL -silent \
  -install_dir /soft/ansys/<version> \
  -licserverinfo 1055:flexlm:license-server.hpc.local

# Create environment module
cat > /soft/modules/ansys/<version> << EOF
#%Module
prepend-path PATH /soft/ansys/<version>/v<ver>/ansys/bin
setenv ANSYS_LICENSE_SERVERS 2325@license-server.hpc.local
EOF
```

Abaqus (FEA)

```
# Run installer
./StartGUI.sh
# Installation dir: /soft/abaqus/<version>

# Set license
export LM_LICENSE_FILE=27000@license-server.hpc.local

# Create module
module load abaqus/<version>
```

Altair HyperWorks (CAE)

```
./altair_<version>_linux64.bin
export ALTAIR_LICENSE_PATH=6200@license-server.hpc.local
module load hyperworks/<version>
```

Gaussian (Computational Chemistry)

```
tar xvf gaussian16.tar.gz -C /soft/gaussian/g16
chmod 750 /soft/gaussian/g16
chgrp gaussian /soft/gaussian/g16

export g16root=/soft/gaussian
export GAUSS_SCRDIR=/scratch/$USER
source $g16root/g16/bsd/g16.profile
```

ICON (Numerical Weather Model)

```
# Install dependencies
dnf install netcdf netcdf-devel hdf5 hdf5-devel -y
module load mpi/openmpi-x86_64

# Build ICON
./configure --with-mpi --with-netcdf --prefix=/soft/icon/<version>
make -j$(nproc)
```

```
make install
```

TensorFlow (Deep Learning)

```
# CPU version  
pip3 install tensorflow
```

```
# GPU version (requires CUDA – see Chapter 12)  
pip3 install tensorflow[and-cuda]
```

```
# Test  
python3 -c "import tensorflow as tf; print(tf.__version__)"
```

Chapter 13: GPU Computing in HPC

13.1 Why GPUs in HPC?

Modern GPUs contain thousands of small cores optimized for floating-point arithmetic, making them ideal for the massively parallel operations common in HPC workloads:

GPU Use Case	Application
Deep Learning Training	TensorFlow, PyTorch
Molecular Dynamics	GROMACS, NAMD, AMBER
CFD Acceleration	Ansys Fluent GPU solver
Seismic Processing	RTM (Reverse Time Migration)
Monte Carlo Simulation	Financial modeling
Image/Signal Processing	Medical imaging

CPU vs GPU comparison:

Feature	CPU (Intel Xeon)	GPU (NVIDIA A100)
Cores	32-64 high-performance	6,912 CUDA cores
Clock Speed	3-4 GHz	1.4 GHz
Memory	512 GB DDR5	80 GB HBM2e
Memory Bandwidth	200 GB/s	2,000 GB/s
Peak FP64	~3 TFLOPS	9.7 TFLOPS
Best For	Serial + parallel tasks	Massively parallel tasks

13.2 NVIDIA GPU Software Stack

The complete software stack required for GPU computing in HPC:

APPLICATIONS TensorFlow PyTorch GROMACS Ansys
HIGH-LEVEL LIBRARIES cuDNN (Deep Learning) TensorRT (Inference) cuBLAS (Linear Algebra) cuFFT (Fast Fourier Transform)
COMMUNICATION NCCL (Multi-GPU collective operations)
CUDA TOOLKIT CUDA Runtime + Compiler (nvcc) CUDA Libraries + Profiler
NVIDIA DRIVER Kernel module + device interface
HARDWARE NVIDIA GPU (A100 / H100 / V100)

13.3 GPU Node Installation Steps

Step 1 — Verify GPU Hardware Detection

```
lspci | grep -i nvidia
# Expected: NVIDIA Corporation A100 80GB PCIe
```

Step 2 — Disable Nouveau Open-Source Driver

The Nouveau driver conflicts with NVIDIA's proprietary driver and must be disabled first:

```
cat > /etc/modprobe.d/blacklist-nouveau.conf << EOF
blacklist nouveau
options nouveau modeset=0
EOF

# Rebuild initramfs
dracut --force
reboot

# Verify nouveau is disabled after reboot
lsmod | grep nouveau # Should return nothing
```

Step 3 — Add NVIDIA Repository

```
dnf config-manager --add-repo \  
  
https://developer.download.nvidia.com/compute/cuda/repos/rhel8/x86_64/cuda-  
rhel8.repo  
dnf clean all
```

Step 4 — Install NVIDIA Driver

```
dnf install nvidia-driver nvidia-driver-libs \  
nvidia-driver-devel nvidia-modprobe -y  
  
# Load driver  
modprobe nvidia  
  
# Verify  
nvidia-smi
```

Expected nvidia-smi output:

```
+-----+  
---+  
| NVIDIA-SMI 525.105.17    Driver Version: 525.105.17    CUDA Version: 12.0  
|  
|-----+-----+-----+  
---+  
| GPU   Name               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr.  
ECC |  
|  0   NVIDIA A100-SXM...    Off | 00000000:01:00.0 Off |  
0 |  
+-----+-----+-----+  
---+
```

Step 5 — Install CUDA Toolkit

```
dnf install cuda-toolkit-12-3 -y  
  
# Add CUDA to system PATH  
cat > /etc/profile.d/cuda.sh << EOF  
export CUDA_HOME=/usr/local/cuda  
export PATH=$CUDA_HOME/bin:$PATH  
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH  
EOF  
  
source /etc/profile.d/cuda.sh  
  
# Verify  
nvcc --version
```


Step 6 — Install cuDNN (Required for Deep Learning) (optional)

cuDNN provides highly optimized primitives for deep neural network operations (convolutions, pooling, normalization):

```
dnf install libcudnn8 libcudnn8-devel libcudnn8-samples -y

# Verify
cat /usr/include/cudnn_version.h | grep CUDNN_MAJOR -A 2
```

Step 7 — Install NCCL (Multi-GPU Communication) (Not in case of single GPU)

NCCL (NVIDIA Collective Communications Library) enables efficient communication between multiple GPUs, both within a node and across nodes via InfiniBand:

```
dnf install libnccl libnccl-devel libnccl-static -y

# For GPU-direct RDMA over InfiniBand
dnf install nv_peer_mem -y
```

Step 8 — Install TensorRT (Inference Optimization)) (optional)

TensorRT optimizes trained neural network models for fast inference on NVIDIA GPUs:

```
dnf install tensorrt tensorrt-devel -y
```

Step 9 — Install GPU Fabric Manager (NVLink Multi-GPU)

Required when multiple GPUs are connected via NVLink (A100, H100 SXM):

```
dnf install nvidia-fabricmanager-525 -y
systemctl enable --now nvidia-fabricmanager

# Verify NVLink topology
nvidia-smi topo --matrix
```

Step 10 — Configure Slurm/OpenPBS for GPU Scheduling

```
# Add to /etc/slurm/slurm.conf on all nodes
GresTypes=gpu

# GPU node definition
NodeName=gpu1 NodeAddr=192.168.100.20 \
  CPUs=64 RealMemory=512000 \
  Gres=gpu:a100:4 State=UNKNOWN

# GPU partition
PartitionName=gpu Nodes=gpu1 Default=NO \
  MaxTime=INFINITE State=UP

# Create /etc/slurm/gres.conf on GPU nodes
cat > /etc/slurm/gres.conf << EOF
AutoDetect=nvml
Name=gpu Type=a100 File=/dev/nvidia[0-3]
```

EOF

```
systemctl restart slurmctld # on master
systemctl restart slurmd   # on GPU node
```

Submit a GPU job:

```
#!/bin/bash
#SBATCH --job-name=GPU_Training
#SBATCH --partition=gpu
#SBATCH --gres=gpu:2
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=gpu_job_%j.out

module load cuda/12.3
module load cudnn/8.9

nvidia-smi
python3 train_model.py --gpus 2 --epochs 100
```

13.4 Complete NVIDIA Software Stack Summary

Package	Purpose	Required For
nvidia-driver	GPU kernel driver and device interface	All GPU workloads
cuda-toolkit	CUDA runtime, compiler (nvcc), libraries	Any CUDA program
libcudnn8	Deep learning primitive library	TensorFlow, PyTorch
libnccl	Multi-GPU collective communications	Distributed training
tensorrt	Neural network inference optimization	Production inference
nvidia-fabricmanager	NVLink fabric management daemon	NVLink multi-GPU (A100/H100 SXM)
cuda-gdb	GPU debugger	CUDA development
nvidia-nsight-compute	GPU kernel profiler	Performance optimization
nvidia-nsight-systems	System-wide GPU/CPU profiler	Bottleneck analysis
nv_peer_mem	GPU-direct RDMA over InfiniBand	MPI + GPU workloads

Chapter 14: High Availability (HA) Clustering

12.1 Introduction

High Availability (HA) clustering is a system design approach used to ensure **continuous operation of critical cluster services** in the event of hardware or software failures. In HPC environments, HA clustering is implemented to eliminate **single points of failure** in key components such as head nodes, login nodes, and storage servers.

By deploying redundant nodes and failover mechanisms, HA clusters maintain system availability even if one component becomes unavailable.

12.2 Importance of HA in HPC Clusters

In a standalone cluster, failure of a critical node (such as the head node or storage server) can interrupt job scheduling, user access, or data availability.

HA clustering provides the following benefits:

Benefit	Description
Fault tolerance	Services continue running if one node fails
Service continuity	Job scheduling and storage remain accessible
Reduced downtime	Automatic failover minimizes manual intervention
Improved reliability	Redundant components prevent system-wide failure

12.3 HA Architecture in HPC Clusters

Head Node 1 (Active)

Head Node 2 (Standby)

|

Login Node 1

Login Node 2

|

Compute Nodes (Multiple)

|

Storage Subsystem

12.4 Redundant Cluster Components

Head Nodes (Master Nodes)

Head nodes manage the overall cluster operations, including job scheduling, resource management, and system monitoring.

Configuration	Description
Head Node 1	Active cluster controller
Head Node 2	Standby node for failover

If the active head node fails, the standby node automatically takes over cluster management functions.

Login Nodes

Login nodes provide user access to the HPC cluster.

Configuration	Description
Login Node 1	Primary user access node
Login Node 2	Secondary login node for redundancy

Users can connect through either node, ensuring continuous access even if one login node fails.

Storage Nodes

Shared storage is critical for HPC workloads, as compute nodes require access to input data and result outputs.

Two common storage configurations are used.

HA NFS Storage

Used in small or mid-size clusters.

Component	Description
NFS Server 1	Primary storage node
NFS Server 2	Secondary node with synchronized data

Failover tools replicate storage and ensure availability.

HA PFS Storage

Large HPC clusters typically use **parallel file systems like Lustre**

Component	Function
MDS (Metadata Server)	Handles file metadata
OSS (Object Storage Server)	Stores file data
MDT	Metadata storage
OST	Object storage targets

Typical HA Deployment:

Component	Count
MDS (Metadata Server)	2
OSS (Object Storage Server)	2 or more

12.5 Failover Mechanisms

HA clusters rely on cluster management tools that detect failures and move services to standby nodes.

Common technologies

Tool	Purpose
Pacemaker	Cluster resource management

12.6 HA Workflow

The following sequence describes typical failover behavior:

1. Cluster monitoring detects node failure.
2. Pacemaker triggers failover procedures.
3. Services are migrated to standby node.
4. Virtual IP is reassigned.
5. Cluster operations continue normally.

High Availability clustering is essential for production HPC environments where uninterrupted operation is required. By deploying redundant head nodes, login nodes, and storage systems along with automated failover mechanisms, HA clustering significantly improves the reliability and availability of HPC infrastructure.

Chapter 15: Summary

15.1 HPC Software Stack Diagram

APPLICATIONS					
Ansys	Abaqus	HyperWorks	Gaussian	ICON	TensorFlow
MPI LAYER					
OpenMPI / Intel MPI / MVAPICH2					
JOB SCHEDULER LAYER					
Slurm / OpenPBS					
IDENTITY & AUTHENTICATION					
FreeIPA / NIS / MUNGE					
STORAGE LAYER					
NFS (/home, /soft, /opt) Lustre (/scratch, /work)					
RAID 0 / 1 / 5 / 6 / 10					
TIME SYNCHRONIZATION					
Chrony / NTP					
CLUSTER MANAGEMENT LAYER					
xCAT / HPCM					
OS LAYER					
RHEL 9.4 / Rocky Linux 8.10					
GPU ACCELERATION (Optional)					
NVIDIA Driver → CUDA → cuDNN → NCCL → TensorRT					
HARDWARE LAYER					
Head Node	Login	Compute	Storage	IB Switch	PDU

15.2 Cluster Setup Checklist

Hardware:

- ☐ BoM verification complete
- ☐ Rack planning documented
- ☐ Rack stacking and mounting complete
- ☐ Power cabling complete (A+B paths)
- ☐ Network cabling complete (management + cluster + IB)
- ☐ Initial power-on successful, no hardware faults

Firmware & BIOS:

- ☐ All firmware updated
- ☐ BIOS configured for performance
- ☐ PXE boot order set

OS & Basic Configuration:

- ☐ Rocky Linux / RHEL installed on all nodes
- ☐ Hostnames set correctly (FQDN)
- ☐ /etc/hosts configured on all nodes
- ☐ Firewall disabled
- ☐ SELinux disabled
- ☐ SSH passwordless root access configured
- ☐ System packages updated

Services:

- ☐ Chrony NTP configured and synced on all nodes
- ☐ NFS server configured and exports verified
- ☐ NFS mounts confirmed on all client nodes
- ☐ RAID arrays created and verified
- ☐ FreeIPA server installed and clients enrolled
- ☐ Users created in IPA

Job Scheduler:

- ☐ MUNGE installed and identical key on all nodes
- ☐ MUNGE service running on all nodes
- ☐ Slurm/PBS installed on all nodes
- ☐ slurm.conf / pbs.conf identical on all nodes
- ☐ slurmctld/pbs_server running on master
- ☐ slurmd/pbs_mom running on compute nodes
- ☐ sinfo / pbsnodes -a shows all nodes idle
- ☐ Test job submitted and completed successfully

Applications:

- ☐ OpenMPI installed and tested
- ☐ Scientific applications installed in /soft
- ☐ Environment modules configured
- ☐ Test application job submitted successfully

GPU (if applicable):

- ☐ Nouveau driver disabled
- ☐ NVIDIA driver installed and nvidia-smi working
- ☐ CUDA toolkit installed

- ☐ cuDNN installed
 - ☐ Slurm GPU resources configured
 - ☐ Test GPU job submitted successfully
-